Numéro d'inscription

Numéro de table

Né(e) le

Nom :

Prénom :

Emplacement QR Code

Filière : PC

Session : 2024

Épreuve de : INFORMATIQUE

Consignes
- *Remplir soigneusement l'en-tête de chaque feuille avant de commencer à composer*
- *Rédiger avec un stylo non effaçable bleu ou noir*
- *Ne rien écrire dans les marges (gauche et droite)*
- *Numéroter chaque page (cadre en bas à droite)*
- *Placer les feuilles A3 ouvertes, dans le même sens et dans l'ordre*

PC5IN

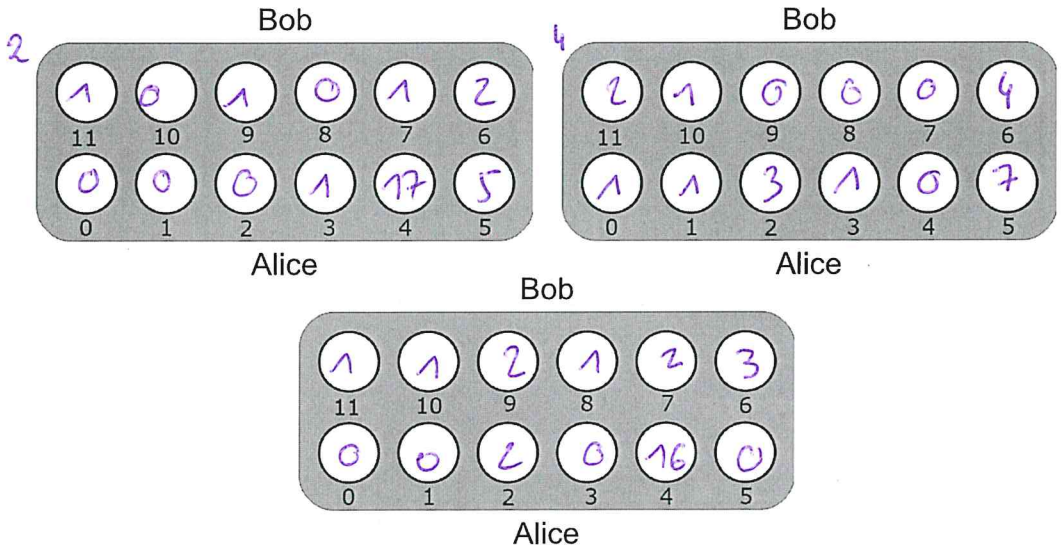
DOCUMENT RÉPONSE

Ce Document Réponse doit être rendu dans son intégralité.

Q1 - Indice et justification

Les cases 2, 4 et 5 car de son côté et non vide

Q2 - Situation possible après le tour d'Alice



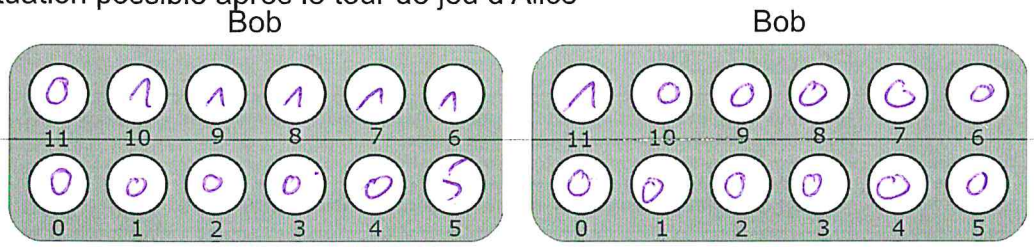
Case 2 : Gain 0

Case 4 : Gain 3+2+3 = 8

Case 5 : Gain 0

NE RIEN ÉCRIRE DANS CE CADRE

Q3 - Situation possible après le tour de jeu d'Alice



Situation 1 : Règle de la famine, Alice ne peut pas jouer.

Situation 2 : Alice sème les 5 graines et récolte 10 graines sur le coté de Bob.

Q4 - Parité de **jeu['n']**. Fonction **tour_joueur1(jeu:dict) → bool**

```
def tour_joueur1(jeu):                # jeu['n'] pair
    return jeu['n'] % 2 == 0          # si à joueur 1 de jouer
```

Q5 - Fonction **tourner_plateau(jeu:dict) → None**

```
def tourner_plateau(jeu):
    p="plateau"
    for i in range(6):                # i de 0 à 5
        jeu[p][i], jeu[p][i+6] = jeu[p][i+6], jeu[p][i]
```

Q6 - Maximum de graines. Bits nécessaires

$4 \times 12 = 48$. On a donc au maximum 48 graines sur une case.

En binaire sur n bits on peut représenter les décimaux dans l'intervalle $[0 ; 2^n - 1]$

Il faut donc 6 bits pour représenter le nombre de graine.

Q7 - Fonction **copie(jeu:dict) → dict**

```
def copie(jeu):
    d = {}
    d["joeur1"] = jeu["joeur1"]
    d["joeur2"] = jeu["joeur2"]
    d["score"] = jeu["score"].copy()
    d["n"] = jeu["n"]
    d["plateau"] = jeu["plateau"].copy()
    return d
```

Q8 - Fonction **deplacer_graines(plateau:[int], case:int) → int**

```
def deplacer_graines(plateau, case):
    nb_graine = plateau [case]
    plateau[case] = 0
    i = case
    while nb_graine >= 0 :
        i += 1    #on passe à la case suivante
        if i >= len(plateau):
            i = 0
        if i != case :    #on ne sème pas sur case
            plateau [i] += 1
            nb_graine -= 1
    return i
```

Q9 - Fonction **case_ramassable(plateau:[int], case:int) → bool**

```
def case_ramassable(plateau, case):
    est_voisin = case >= 6
    est_ramassable = 2 <= plateau[case] <= 3
    return est_voisin and est_ramassable.
```

Numéro d'inscription

Numéro de table

Né(e) le

Nom :

Prénom :

Emplacement QR Code

Filière : PC

Session : 2024

Épreuve de : INFORMATIQUE

Consignes
- *Remplir soigneusement l'en-tête de chaque feuille avant de commencer à composer*
- *Rédiger avec un stylo non effaçable bleu ou noir*
- *Ne rien écrire dans les marges (gauche et droite)*
- *Numéroter chaque page (cadre en bas à droite)*
- *Placer les feuilles A3 ouvertes, dans le même sens et dans l'ordre*

Q10 - Fonction récursive **ramasser_graines(plateau:[int], case:int) → int**

```
def ramasser_graines(plateau, case):
    if not case_ramassable(plateau, case):
        return 0
    else:
        s = plateau[case]
        plateau[case] = 0
        return s + ramasser_graines(plateau, case-1)
```

NE RIEN ÉCRIRE DANS CE CADRE

Q11 - Fonction **test_famine(plateau:[int], case:int) → bool**

```
def test_famine(plateau, case):
    copie = plateau.copy()
    arrive = deplacer_graine(copie, case)
    gain = ramasser_graines(copie, arrive)
    for i in range(6,12):
        if copie[i] != 0:
            return True
    return False
```

Q12 - Expression de la condition test

```
0<= case <= 6 and plateau[case] != 0 and condition 3
```

Q13 - Fonction **cases_possibles(jeu:dict) → [int]**

```
def cases_possibles(jeu):
    possible = []
    for i in range(6):
        if test_case(jeu["plateau"], i):
            possible.append(i)
    return possible
```

Q14 - Fonction **tour_suivant(jeu:dict) → bool**

```
def tour_suivant(jeu):
    if jeu['score'][0] >= 25 or jeu['score'][1] >= 25:
        return False
    if jeu['n'] >= 100
        return False
    if (jeu['score'][0] + jeu['score'][1]) >= 45:
        return False
    if len(cases_possible(jeu)) == 0:
        return False
    return True
```

Q15 - Instruction 1, instruction 2, condition 1 et instruction 3

Instruction 1

```
case_arrive = deplacer_graines(plateau, case)
```

Instruction 2

```
graines_gagnees = ramasser_graines(plateau, case_arrive)
```

Condition 1

```
tour_joueur1(jeu)
```

Instruction 3

```
jeu['n'] += 1
```

Q16 - Fonction **gagnant(jeu:dict) → str**

```
def gagnant(jeu):
    if tour_joueur1(jeu):
        jeu['score'][0] += sum(jeu['plateau'][:6])
        jeu['score'][1] += sum(jeu['plateau'][6:])
    else:
        jeu['score'][0] += sum(jeu['plateau'][6:])
        jeu['score'][1] += sum(jeu['plateau'][:6])
    if jeu['score'][0] == jeu['score'][1]:
        return "égalité"
    elif jeu['score'][0] > jeu['score'][1]:
        return jeu['joueur1']
    else:
        return jeu['joueur2']
```